Домашнее задание 4. Каталог с сервера: GET и POST

Подключаем каталог к настоящему серверу: элементы загружаются по сети GET-запросом, а новые добавляются POST-запросом и сохраняются на сервере.

Сервером будет MockAPI — бесплатный сервис, в котором за пару минут можно сделать свой учебный API под тему своего каталога. Как им пользоваться — подробно в разделе 2.

Дедлайн: понедельник, 19 октября, 23:59 по МСК.

1. Что нужно сделать

Основа — ваш каталог из ДЗ 3: та же тема, модель, лайки и фильтр «Только избранные».

Часть 1. Сервер на MockAPI

1. Создайте свой API на MockAPI и заполните его 8 и более элементами своего каталога (инструкция — в разделе 2).
2. Откройте адрес ресурса в браузере и проверьте, что приходит список элементов.

Часть 2. Модель

1. Поля модели совпадают с полями в MockAPI, а **типы полей совпадают с тем, что приходит с сервера**.
2. В модели есть `factory Book.fromJson(Map<String, dynamic> json)` (имя класса — ваше) и новый метод `Map<String, dynamic> toJson()` — обратное превращение модели в `Map`, чтобы отправить её на сервер.

Часть 3. Загрузка каталога (GET)

1. Каталог загружается с сервера при открытии экрана (запрос запускаем в `initState`). Константный список элементов из ДЗ 3 удалите.

2. Пока идёт загрузка, на экране индикатор `CircularProgressIndicator`.
3. Если запрос не удался (нет интернета, неверный адрес, статус не 200) — текст ошибки и кнопка Обновить(заново запускает запрос списка).

Часть 4. Добавление элемента (POST)

1. Над списком — форма добавления элемента.
2. Обязательное поле в форме одно — «Название» (`TextField` с `TextEditingController`). Другие поля можно добавить по желанию. По желанию это поле(поля) можно сделать сворачиваемыми
3. По кнопке «Отправить» элемент уходит на сервер POST-запросом. Поля, которые пользователь не вводит, заполните значениями по умолчанию. `id` не отправляйте — его выдаёт сервер.
4. Пустое название не отправляется.
5. Новый элемент берите из ответа сервера (в нём уже есть `id`) и добавляйте в начало списка. После отправки поле очищается.
6. Если отправка не удалась, покажите сообщение об ошибке. Приложение не должно падать.
7. После перезапуска приложения добавленный элемент на месте — значит, сервер действительно его сохранил.

Лайки и фильтр «Только избранные» продолжают работать, но локально: на сервер лайк не отправляем.

Подсказки:

- Пакет `http` подключается командой в корне проекта: `flutter pub add http`. В файле нужны импорты `import 'package:http/http.dart' as http;` и `import 'dart:convert';`.
- GET-запрос и превращение ответа в список моделей:

```
final response = await http.get(Uri.parse('$baseUrl/books'));
if (response.statusCode != 200) throw Exception('Статус ${response.statusCode}');
final list = jsonDecode(response.body) as List;
final books = list.map((e) => Book.fromJson(e)).toList();
```

- POST-запрос: тело — строка JSON, заголовок `Content-Type` обязателен. Успешное создание — статус 201, а не 200.

```
final response = await http.post(
  Uri.parse('$baseUrl/books'),
  headers: {'Content-Type': 'application/json; charset=UTF-8'},
  body: jsonEncode(book.toJson()),
);
if (response.statusCode != 201) throw Exception('Статус ${response.statusCode}');
final created = Book.fromJson(jsonDecode(response.body));
```

- Чтобы не отправлять `id`, просто не добавляйте его в `toJson()`. Все остальные поля модели отправляйте: если какого-то поля не окажется на сервере, при следующей загрузке `fromJson` упадёт на `null`.
- Значения по умолчанию могут быть такими: `description: ''`, `tags: []`, `details: {}`.
- Ошибка `type 'String' is not a subtype of type 'int'` значит, что тип поля в модели не совпадает с тем, что пришло с сервера. Откройте адрес ресурса в браузере и сравните.
- `TextEditingController` создайте в `State` и освободите в `dispose()`, как в лекции.
- Если `TextField` стоит внутри `Row` (например, рядом с кнопкой), оберните его в `Expanded`, иначе будет ошибка «unbounded width».
- По желанию: кнопка «+» в шапке экрана (или в `AppBar`), которая показывает и скрывает форму. Это флаг `bool` и `setState`, как у фильтра «Только избранные» в ДЗ 3, а форму показываем через `if` прямо в списке `children`:

```
bool _showForm = false;
IconButton(
  icon: Icon(_showForm ? Icons.close : Icons.add),
  onPressed: () => setState(() => _showForm = !_showForm),
)
```

```
// в Column экрана:
if (_showForm) _buildForm(),
```

Запустите проект и проверьте: каталог загружается с сервера, новый элемент добавляется и остаётся после перезапуска. После этого переходите к сдаче.

2. Как пользоваться MockAPI

MockAPI — сервис для учебных и тестовых API. Вы описываете, какие поля есть у элемента, а он сам создаёт сервер с адресами для чтения, создания, изменения и удаления элементов. Данные хранятся на сервере MockAPI, поэтому то, что вы отправили POST-запросом, не пропадёт после перезапуска приложения.

Два понятия, которые встретятся дальше. **Проект** — это ваш «сервер» со своим адресом. **Ресурс** — «таблица» внутри проекта (например, `books`): у него свой адрес и свой список элементов.

Шаг 1. Зарегистрируйтесь

Откройте mockapi.io и войдите через GitHub — тем же аккаунтом, что и для домашек. Бесплатного тарифа для задания хватает.

Шаг 2. Создайте проект

Нажмите кнопку создания нового проекта (New project или «+»). Название — любое, например `catalog`. Поле API prefix можно оставить пустым.

Откройте созданный проект. Сверху будет его адрес вида `https://<ваш-токен>.mockapi.io` (если указали prefix — он будет в конце адреса). Это базовый адрес вашего API, он понадобится в коде.

Шаг 3. Создайте ресурс

В проекте нажмите New resource. В имени ресурса напишите тему каталога латиницей во множественном числе, например `books`, `dogs`, `movies`. Имя станет частью адреса: `https://<ваш-токен>.mockapi.io/books`.

Ниже — схема (Schema), то есть поля одного элемента. MockAPI сразу добавляет поля-примеры (`id`, `createdAt`, `name`, `avatar`). Поле `id` оставьте,

остальные удалите и добавьте поля своей модели. Например, для файла-образца это:

```
title        String
subtitle     String
category     String
description  String
image        String
year         Number
tags         Array
```

Тип выбирайте по значению: текст — String, число — Number, список — Array. Имена полей должны в точности совпадать с ключами в JSON и в `fromJson`. Нажмите Create.

Шаг 4. Заполните ресурс данными

Рядом с ресурсом нажмите Data — откроется редактор с JSON-массивом элементов. Удалите всё, что там есть, вставьте свой массив вида `[ {...}, {...} ]` и сохраните.

Где взять данные. Посмотрите https://github.com/McLavrushka/flutter_course_inno/blob/main/assets/data/example.json — в нём 3 элемента каталога книг, и видно, как записываются строки, числа и списки. Составьте такой же массив из 8 и более элементов своего каталога одним из способов:

- Сами — перепишите элементы из кода ДЗ 3 в JSON по образцу.
- С помощью нейросети — пришлите ей файл-образец и свой список элементов из кода ДЗ 3, например так: «Преврати мой список элементов каталога в JSON-массив по этому образцу. Поля: ... Тексты на русском, только реальные факты. Верни только JSON, без пояснений».
- Попросите меня — напишите в личные сообщения и пришлите свой список элементов - я сгенерирую JSON.

Если сохранить не получается, проверьте, что JSON целиком скопирован: он начинается с `[` и заканчивается `]`.

Подсказки для вашего каталога:

- **Фото из assets** остаются в проекте, а на сервере храните только путь к файлу строкой, как в образце: `"image": "assets/images/dune.jpg"`. В карточке, как и раньше, `Image.asset(item.image)`. Для новых элементов из формы подставьте картинку-заглушку по умолчанию (например, `assets/images/placeholder.png`) или цветную обложку.
- **Цвет обложки** храните строкой: `"color": "FFB74D"`, а в `fromJson` превращайте в цвет: `Color(int.parse('FF${json['color']}', radix: 16))`.
- **Категорию-enum** храните строкой, как `category` в образце: `"category": "classic"`. В `fromJson` — `Category.values.byName(json['category'])`, в `toJson` — `category.name`. Значения в JSON должны совпадать с именами вариантов enum.
- Если JSON сделала нейросеть, проверьте, что имена полей совпадают с вашим `fromJson`, а файл — массив, который начинается с `[` и заканчивается `]`.

Шаг 5. Проверьте API в браузере

Откройте в браузере адрес ресурса, например `https://<ваш-токен>.mockapi.io/books`. Должен прийти JSON-массив ваших элементов. Один элемент открывается по адресу с его id: `.../books/1`.

Если вместо русских букв в браузере «кракозябры» — это особенность браузера: он не угадал кодировку. Данные в порядке, в приложении текст будет нормальным. Убедиться можно в Data на MockAPI — там текст отображается правильно.

Шаг 6. Подставьте адрес в код

Вынесите базовый адрес в константу в начале файла экрана:

```
const baseUrl = 'https://<ваш-токен>.mockapi.io';
```

В задании понадобятся два адреса:

- `GET $baseUrl/books` — получить список всех элементов;

- `POST $baseUrl/books` — создать элемент. В теле — JSON без `id`, в ответе — созданный элемент уже с `id`.

Остальные запросы (получить один элемент, изменить, удалить) MockAPI тоже умеет, но в этом задании они не нужны.

Если что-то пошло не так

- Испортили данные — откройте Data и вставьте массив заново.
- Данные в MockAPI публичные: любой, у кого есть ссылка, может их прочитать и изменить. Не храните там пароли и личные данные.

3. Как сдать работу через GitHub

Работаем в том же репозитории и проекте, каталог по-прежнему живёт в `lib/`.

Шаг 1. Смерджите hw3

Откройте Pull Request по ДЗ 3 на GitHub, нажмите зелёную кнопку Merge pull request, затем Confirm merge. Так в новой ветке будет код из ДЗ 3, который мы продолжаем. Смерженный Pull Request по-прежнему открывается по ссылке.

Шаг 2. Обновите main

Откройте терминал в корне проекта (там, где лежит `pubspec.yaml`) и выполните:

```
git checkout main
git pull origin main
```

Первая команда переключает вас на ветку main, вторая подтягивает её последнюю версию с GitHub — уже вместе со смердженным ДЗ 3.

Шаг 3. Создайте ветку hw4

```
git checkout -b hw4
```

Команда создаёт ветку hw4 и сразу переключает вас на неё.

Шаг 4. Подключите пакет http

```
flutter pub add http
```

Команда добавит пакет в `pubspec.yaml`. Эти изменения тоже попадут в коммит — так и нужно.

Шаг 5. Реализуйте задание

Создайте API на MockAPI (раздел 2), обновите модель, сделайте загрузку и добавление элементов. Запуск обычный:

```
flutter run
```

Шаг 6. Закоммитьте и отправьте на GitHub

```
git add .
git commit -m "hw4: get and post with mockapi"
git push -u origin hw4
```

После этого на GitHub появится ветка hw4 с вашим решением.

Шаг 7. Откройте Pull Request в main

Зайдите на страницу репозитория на GitHub. Обычно сразу видна жёлтая плашка «hw4 had recent pushes» с кнопкой Compare & pull request, нажмите её. Если плашки нет, откройте вкладку Pull requests → New pull request, слева укажите base: main, справа compare: hw4.

Проверьте, что сравниваются именно main и hw4, добавьте короткое название (например «hw4») и нажмите Create pull request. Сами этот Pull Request не мерджите: он должен остаться открытым.

Шаг 8. Запишите демо

Запишите видео экрана на 30-40 секунд:

- запуск приложения — видно индикатор загрузки и каталог с сервера;
- ввод названия, отправка — новый элемент появился в начале списка;
- приложение полностью закрыто и запущено заново — добавленный элемент на месте.

Шаг 9. Сдайте работу в LMS

До дедлайна загрузите на платформу:

- видео с демо: загрузка, добавление, перезапуск;
- ссылку на открытый Pull Request. Скопируйте адрес страницы Pull Request из адресной строки браузера, он выглядит как `https://github.com/ваш-логин/название/pull/4`.

4. Важно

- Репозиторий обязательно должен быть публичным (Public), иначе я не смогу открыть ваш код.
- Pull Request по hw4 должен быть открыт к моменту проверки, не мерджите его сами.
- Не удаляйте и не переименовывайте ветку main, вся работа идёт в отдельных ветках вроде hw4.
- Если на любом шаге команда выдаёт ошибку, не удаляйте и не пересоздавайте всё с нуля, пришлите мне текст ошибки, разберёмся.